C:\hpc-23-feb-2026\.\36\eg213.cpp

```
#include<iostream>
#include<oneapi/tbb.h>
using namespace tbb;
using namespace std;
int main()
{
int x[]={10,20,30,40,50,60,70,80,90,100};
int idx=0;
// create first filter : serial_in_order
auto serial_input_filter=make_filter<void,int>(filter_mode::serial_in_order,[&](flow_control &fc)->i
nt{
if(idx<10)
{
return x[idx++];
}
fc.stop();
return 0;
});
// create second filter : parallel
auto parallel_filter=make_filter<int,int>(filter_mode::parallel,[](int num)->int{
return num*num;
});
// create third filter : serial_in_order
auto serial_output_filter=make_filter<int,void>(filter_mode::serial_in_order,[](int num){
cout<<num<<endl;
});
// create pipeline
parallel_pipeline(4,serial_input_filter & parallel_filter & serial_output_filter);
cout<<"Job done"<<endl;
return 0;
}
```

C:\hpc-23-feb-2026\.\36\eg214.cpp

```
#include<iostream>
#include<oneapi/tbb.h>
using namespace tbb;
using namespace std;
int main()
{
int x[200];
int idx;
for(idx=0;idx<200;++idx) x[idx]=idx+1;
idx=0;
// create first filter : serial_in_order
auto serial_input_filter=make_filter<void,int>(filter_mode::serial_in_order,[&](flow_control &fc)->i
nt{
if(idx<200)
{
return x[idx++];
}
fc.stop();
return 0;
});
// create second filter : parallel
auto parallel_filter=make_filter<int,int>(filter_mode::parallel,[](int num)->int{
num=num*10;
num=num/10;
return num;
});
// create third filter : serial_in_order
auto serial_output_filter=make_filter<int,void>(filter_mode::serial_in_order,[](int num){
cout<<num<<endl;
});
// create pipeline
parallel_pipeline(4,serial_input_filter & parallel_filter & serial_output_filter);
cout<<"Job done"<<endl;
return 0;
}
```